# Chapter 18 worked solutions • The Linux system

## OS18-E01 • Fork and exec

1. Child task, mm, and page tables already exist. Private user frames initially share read-only mappings;reading needs no content copy. Immediate exec replaces those mappings, avoiding copies soon discarded. Metadata cost remains.
2. In this single-thread case, child PID persists while code, data, stack, entry are replaced. Parent remains independent. Multithreaded exec has additional thread termination rules, not another ordinary fork.
3. Early failure returns to old code;record errno and explicitly _exit as appropriate so parent wait observes failure. After v3.2 ELF flush_old_exec, some failures deliver fatal signals without restorable old image. Parent distinguishes ordinary exit from signal termination. Identify task/address space at every step.

## OS18-E02 • Module references

1. No. Both references can still execute module code.
2. Block new legal entry/references, complete/cancel I/O with DMA stopped, close handles, and wait for actual worker/timer/RCU protocols. Free only when references and callbacks no longer use code. There is no universal one-RCU-wait rule.
3. No. Signatures support provenance/integrity policy, not pointer/DMA lifetime. Universal modern signature enforcement is not assumed for v3.2. References, I/O, and quiescence are separate conditions.

## OS18-E03 • VFS and page cache

1. The call observed those cache bytes, not necessarily identical persistent media.
2. Dirty pages or block/controller caches may remain unwritten. Ordinary reads do not require durable writes.
3. Appropriate flush/commit can require relevant dirty data/metadata to reach its promised persistent layer. Exact guarantees depend on filesystem/API/device contracts. Visibility, writeback, and durability differ.

# Chapter 18 worked solutions • The Linux system (continued)

## OS18-E04 • Credentials and regular-file DAC

1. In question order, read/write/execute are allow/deny/deny;allow/allow/deny;deny/deny/deny;allow/deny/deny. Ordinary DAC selects the caller's class before capability fallback. DAC_OVERRIDE cannot supply regular-file execute when every x bit is0.
2. Mode 0401 makes S_IXUGO nonzero, permitting its execute DAC override. Noexec, ELF format, and other policies still apply. This is one regular-file DAC layer, not unrestricted root success.
3. No. IDs become1000/1000/0; saved0 permits restoration. Permanent dropping also addresses saved IDs, groups, capability sets, and privileged handles. UID values alone do not prove a sandbox. Derive from actual class/mode/capability branches.

## OS18-E05 • Timeout, EIO, and late completion

1. No. Timeout means no observed completion by a deadline. If F40 is reassigned, late DMA corrupts its new owner. Obtain cancel/reset/other hardware-stop evidence before unmapping.
2. Publish exactly one terminal notification, errorEIO, transferred0 because no earlier successful bytes exist. Omitting error notification strands the waiter and reference cleanup.
3. Same-generation terminal duplicates cannot notify/free again. Old3 cannot complete new4. Identity checking does not replace stopping DMA.
4. No. Consumer can still read/copyF40. Reuse requires DMA stopped, terminal processing completed, and all content references released. Notification and reclamation are different events;retain ownership, count, bytes, and reference state.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Chapter 18 worked solutions • The Linux system (continued)

## OS18-E06 • Clone sharing and ELF zero-fill

1. Counting parent+child, fork has2 tasks,2 mm,2 fd tables, but both original fd4 entries retain O1. Thread-like clone has2 tasks,1 mm,1 table, sameO1, sharedTGID. Neither shares the task_struct object itself.
2. Both fail EINVAL. THREAD requires SIGHAND;SIGHAND requires VM. Flags are not independent booleans, and shared VM alone does not imply sharedTGID.
3. Page0x400000 contains4096 file bytes;0x401000 contains2048 file+2048 zero;0x402000 contains4096 zero. Totals6144+6144=12288 virtual B. VMA states source/permissions;residency can wait for faults.
4. Validation failure before destroying the old image can return normally. After v3.2 flush_old_exec, restoration is not guaranteed;some errors terminate bySIGKILL/SIGSEGV. Successful single-thread exec preservesPID and replaces image rather than forking. Separate task/mm/fd/O1, virtual/resident, and commit boundaries.

## OS18-E07 • IRQ reentry and unload races

1. Preemption disable blocks task rescheduling, not IRQs. Handler spins on L while its interrupted holder cannot release it.
2. Local masking does not stop CPU1. All participating contexts use an atomic spinlock, with irq-save/restore for local reentry. NMI/nonparticipants remain outside the guarantee.
3. Counts5,5,2,0,0. GOING blocks entry. At zero, wait for actual worker, timer, RCU, DMA protocols to quiesce, then free. No universal single-RCU shortcut exists.
4. Another CPU can acquire a reference between zero-check and withdrawal, then execute freed code. State checking/reference acquisition need one lifetime protocol before draining. Prove local reentry, SMP exclusion, and last-reference races separately.

# Chapter 18 worked solutions • The Linux system (continued)

## OS18-E08 • TCP segmentation and latency causes

1. MSS1460;6000=4 • 1460+160 gives five payloads. Headers5 • 40=200, L3 bytes6200. Ethernet, ACK, retransmissions, and offload change denominators.
2. No. The API accepted bytes. TCP acknowledgment concerns peer protocol state, not application consumption or remote durable storage.
3. Queue2 ms, dispatch-to-transmit1 ms, transmit-to-ACK10 ms, total13. These linked timestamps support the decomposition;overlapping aggregate layer averages cannot be arbitrarily added.
4. No. Counters lack same-flow timing links. Collect request/flow timestamps, queue entry/exit, drop, ACK, retransmission evidence or controlled interventions to distinguish queueing, loss, reordering. State application/wire/ACK/completion denominators.

## OS18-E09 • Pipe EOF, signals, and semop

1. Parent's retained writer prevents EOF. Empty buffer means blocking/EAGAIN until all writers close;only then does read return0. Fork does not close unused endpoints automatically.
2. No. Repeated blocked standard signals can coalesce into one pending event. Realtime signals can queue/hold limited payload through corresponding APIs, with queue/handler constraints.
3. No. Atomic batching publishes no partial decrement while waiting. Nonblocking failure also cannot partially subtract. S0 cannot be permanently consumed first.
4. Consumer sees false;producer sets true/wakes before consumer enqueues;consumer then sleeps and loses the notification. Establish wakeable waiting state, then recheck;skip sleep if true and recheck after wake. Actual kernel primitives supply locking/ordering too. Stream data, writer lifetime, pending signals, and semaphore transactions are different state models.

# Chapter 18 worked solutions • The Linux system (continued)

## Verification code appendix: fork_cow_model.py

```
"""Linux-style private-page fork model, not a kernel implementation.
Mapping counts exclude other kernel references. No huge pages or MAP_SHARED.
"""
from dataclasses import dataclass
from collections import Counter
import json

@dataclass
class PTE:
    frame:int
    writable:bool
    cow:bool=False

@dataclass
class MM:
    name:str
    table:dict[int,PTE]

@dataclass
class Task:
    pid:int
    mm:MM
    state:str='ready'

class Model:
    def __init__(self):
        self.frames={17:bytearray([41])+bytearray(4095)}
        self.parent=Task(100,MM('M100',{0x4000:PTE(17,True)}))
        self.tasks=[self.parent];self.faults=0;self.copied_bytes=0
    def fork(self,clone_vm=False):
        if clone_vm: mm=self.parent.mm
        else:
            table={}
            for va,pte in self.parent.mm.table.items():
                pte.writable=False;pte.cow=True
                table[va]=PTE(pte.frame,False,True)
            mm=MM('M101',table)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Chapter 18 worked solutions • The Linux system (continued)

```
        child=Task(101,mm);self.tasks.append(child);return child
    def mappings(self):
        distinct={id(t.mm):t.mm for t in self.tasks}
        return dict(Counter(p.frame for mm in distinct.values() for p in mm.table.values()))
    def write(self,task,value):
        pte=task.mm.table[0x4000]
        if not pte.writable:
            assert pte.cow
            self.faults+=1
            if self.mappings()[pte.frame]>1:
                self.frames[23]=bytearray(self.frames[pte.frame])
                self.copied_bytes+=4096;pte.frame=23
            pte.writable=True;pte.cow=False
        self.frames[pte.frame][0]=value
    def snapshot(self,label):
        return {'stage':label,'tasks':
[{'pid':t.pid,'mm':t.mm.name,'frame':t.mm.table[0x4000].frame,'W':int(t.mm.table[0x4000].writ
able),'COW':int(t.mm.table[0x4000].cow),'x':self.frames[t.mm.table[0x4000].frame][0]} for t
in
self.tasks],'mapped_PTE_counts':self.mappings(),'faults':self.faults,'copied_data_bytes':self
.copied_bytes}

def main():
    m=Model();trace=
[m.snapshot('before')];child=m.fork();trace.append(m.snapshot('after_fork'))
    assert child is not m.parent and child.mm is not m.parent.mm
    assert child.mm.table is not m.parent.mm.table
    assert child.mm.table[0x4000] is not m.parent.mm.table[0x4000]
    child.state='blocked';assert m.parent.state=='ready'
    assert m.copied_bytes==0 and m.mappings()=={17:2}
    m.write(child,42);trace.append(m.snapshot('child_first_write'))
    assert m.frames[17][0]==41 and m.frames[23][0]==42
    assert m.mappings()=={17:1,23:1} and m.copied_bytes==4096 and m.faults==1
    shared=Model();other=shared.fork(clone_vm=True)
    assert other is not shared.parent and other.mm is shared.parent.mm
    shared.write(other,42);assert shared.frames[17][0]==42 and shared.faults==0
    wrong=Model();bad_child=wrong.parent;bad_child.pid=101
    assert wrong.parent.pid==101
    print(json.dumps({'fork_trace':trace,'CLONE_VM_parent_value':shared.frames[17]
[0],'metadata_alias_counterexample_parent_pid':wrong.parent.pid},indent=2))
if __name__=='__main__':main()
```

# Chapter 18 worked solutions • The Linux system (continued)

## Verification code appendix: module_lifecycle.py

```
"""State-machine check for safe module unload.

Run:
    python module_lifecycle.py
"""

from dataclasses import dataclass


@dataclass
class Module:
    active_handles: int = 0
    queued_requests: int = 0
    loaded: bool = True
    accepting: bool = True
    callbacks: int = 0

    def can_unload(self) -> bool:
        return (self.loaded and not self.accepting and self.active_handles == 0
                and self.queued_requests == 0 and self.callbacks == 0)

    def acquire_handle(self) -> None:
        if not self.accepting:
            raise RuntimeError("new entries stopped")
        self.active_handles += 1

    def begin_unload(self) -> None:
        self.accepting = False

    def unload(self) -> None:
        if not self.can_unload():
            raise RuntimeError("module still referenced")
```

# Chapter 18 worked solutions • The Linux system (continued)

```
        self.loaded = False


def main() -> None:
    module = Module(active_handles=3, queued_requests=2)
    module.begin_unload()
    try:
        module.unload()
    except RuntimeError:
        pass
    else:
        raise AssertionError("unload must fail while referenced")
    try:
        module.acquire_handle()
    except RuntimeError:
        pass
    else:
        raise AssertionError("GOING must reject a new reference")
    module.active_handles = 0
    module.callbacks = 1
    module.queued_requests = 0
    assert not module.can_unload(), "last callback still executes module code"
    module.callbacks = 0
    module.unload()
    assert not module.loaded
    print("module unloaded after quiescence")


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

### References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study